

## GIK2NV DSTM Lab 1: PLSQL Packages

**Post a link to your movie presentation below:**

- [Hand in a MP4-file in the course room.](#)

During this lab, you will create a PLSQL package that handles customer accounts for secure password storage. Furthermore, there will be functions for authentication and password management.

Use the eService: <https://livesql.oracle.com/>

**NOTE!** This lab is a group assignment..

**To be approved in the lab, the following is required:**

1. All code should work.
2. The code should be neat and easy to read according to clean code principles.
3. [You should record a movie with sound where you explain the code and show that the code works. If you like, you can upload the movie to YouTube and enter the link to the movie in the lab report. Make sure the link really works before submitting the lab.](#)
4. [Or hand in a MP4-file in the course room.](#)
5. Submit this lab with your code in the course room in Canvas.  
Assignments: Lab 01: PLSQL Packages
6. [If enrolled in a group, it is enough if one of the students in a group hands in the lab report.](#)

## The task

Start by creating the following objects: a customer table with constraints and a sequence for primary key values in the table. Do that by copy and paste.

```
-----START COPY AND PASTE-----  
create table customer (  
    custid      number(10)      not null,  
    username    varchar2(30)    not null,  
    password    varchar2(40)    not null  
);  
  
alter table customer add (  
    constraint customer_pk primary key (custid)  
);  
  
alter table customer add (  
    constraint customer_uk unique (username)  
);  
  
create sequence customer_seq;  
-----END COPY AND PASTE-----
```

When we add customers to the customer table, we do not want to store the password in plain text. This is to protect us from security breaches.

The following function has been created to increase security:

```
-----START COPY AND PASTE-----  
create or replace function get_hash(  
    p_username  in  varchar2,  
    p_password  in  varchar2)  
    return varchar2  
    as  
    l_salt varchar2(30) := 'customerSalt';  
begin  
    return dbms_crypto.hash(utl_raw.cast_to_raw(upper(p_username) ||  
l_salt || upper(p_password)), dbms_crypto.hash_sh1);  
end;  
-----END COPY AND PASTE-----
```

If we call the function `get_hash`, it may look like this:

```
select get_hash('HansRosenball','ScruffyBeard')
from dual;
```

```
GET_HASH('HANSROSENBALL','SCRUFFYBEARD')
-----
8E117C89610BE7FF242BCD6B0DB2ABC35DF72441
```

Instead of storing the password as 'ScruffyBeard' in the database table's column password for customer with username = 'HansRosenball', the encrypted string is stored: '8E117C89610BE7FF242BCD6B0DB2ABC35DF72441'

### Task 1

Create a procedure named `add_customer`. This procedure should add a new row to the customer table. Use the `customer_seq` sequence to fix ID values and the `get_hash` function to store the password securely.

Test the procedure!

### Task 2

Create a function named `get_login`. The function must return 1 if the login is successful and 0 if it is wrong. You must of course send username and password to the function in order for it to work.

Test the function!

### Task 3

Create a procedure named `change_password`. The procedure should change the password for a specific customer. You call the procedure like this:

```
Begin
Change_password('username','old_password','new_password');
End;
```

NOTE! The procedure should only update the password if the `old_password` is correct.

Test the procedure!

#### Task 4

If everything works it is time to put all the sub-programs in a PLSQL package. Create the Spec. and the body. The name of the package should be customer\_security.

Test that everything works.

Remember:

```
Begin  
customer_security.add_customer('username', 'password');  
End;
```

**NOTE!** When you are recording your movie-presentation you must show that everything works within your packet. I don't want to see the testing of the sub.programs outside the packet.

Vijay Paidi  
vpp@du.se